How Secure Is Your Base? A Recent Multi-Scanner Census of Linux Operating-System Images on Docker Hub

Anonymous

[1]Anonymous Institution

Abstract. *Every container starts from an operating-system (OS) base image, yet these are usually measured with a single scanner. We present a recent multi-scanner census of Docker Hub's Linux OS base images:* ***5,606*** *unique* `amd64` *images across* ***20*** *repositories, scanned with* ***14*** *open-source tools. We find that vulnerability posture varies by an order of magnitude across distributions and is driven by the distribution it belongs to and by image age, not by image size or popularity; and that about one in five historically-published images can no longer be pulled by a modern Docker (legacy manifest schema). As a secondary result, the four package-vulnerability engines barely agree, and the high raw secret and malware detection rates collapse to near zero under manual validation: the posture a single tool reports is largely a property of that tool. We publicly release this multi-scanner dataset, to our knowledge the first focused on OS base images, together with the full pipeline.*

1. Introduction

Containers are now a default unit of software delivery: Docker reports more than 20 million developers[1] and, in its 2025 survey, 92% of IT professionals using containers[2]. Underneath them sits Docker Hub, the largest public registry, with on the order of 8.3 million image repositories and hundreds of billions of cumulative pulls[3]. Every one of those images ultimately begins with a single line, `FROM` an operating-system (OS) base image, and a handful of bases are the common root of nearly all of them: `alpine`, `ubuntu`, `debian` and `busybox` each record more than a billion pulls, and Debian, Alpine and Ubuntu dominate the base-image choice of the ecosystem[4]. Because a flaw in a base image is inherited, unchanged, by every image built on it [Shu et al. 2017], the security posture of these few OS bases is a property the whole ecosystem inherits.

That inherited posture is poor and consequential. Base images are the root of the software supply chain, so their known vulnerabilities are a supply-chain risk in their own right, alongside the malicious-injection attacks that dominate headlines (projected to cost USD 60 billion in 2025[5] and predicted by Gartner to hit 45% of organisations by 2025[6]). And the inherited risk is large: popular Docker Hub images carry hundreds of known vulnerabilities on average, many of them years old[7], against a backdrop of record CVE disclosure (over 48,000 CVEs in 2025)[8]. Worse, some of the most-pulled OS bases are

[1]Docker, 2024 report
[2]Docker, 2025 survey
[3]Docker Index
[4]Anchore, 2024
[5]Cybersecurity Ventures
[6]Gartner, 2022
[7]NetRise, 2024
[8]CVE Details

no longer maintained (every official `centos` tag reached end-of-life in June 2024) yet remain in heavy use.

Despite this central role, the security posture of OS base images has usually been characterised with a *single* scanner and on samples that age quickly: the large-scale Docker Hub measurements have typically used one detector [Shu et al. 2017, Zerouali et al. 2019, Liu et al. 2020, Shi et al. 2025], and the studies that *do* compare scanners have done so on focused samples [Mills et al. 2023, Boles et al. 2024]. No recent work measures the Linux OS base images specifically, with a heterogeneous multi-scanner battery, as a current census. This paper fills that gap.

Contributions. We present (i) a recent multi-scanner census of Docker Hub's Linux OS base images (Section 3); (ii) a per-distribution posture, a staleness-vulnerability relationship, and the finding that vulnerability load is driven by age rather than image size or popularity (Sections 4.1, 4.2, 4.5); (iii) an image-longevity finding (Section 4.4); (iv) as a secondary result, a four-engine divergence measurement on the OS-package layer (Section 4.3); and (v) a public multi-scanner dataset, to our knowledge the first focused on OS base images: the per-image findings of all 14 scanners over the census, released with the full pipeline so the corpus can be re-analysed under other metrics or scanners without repeating the costly scan.

2. Related Work

Table 1 positions our work against prior measurements along six axes.

Table 1. Prior OS/base-image and container-scanner measurements.

Study	Images	Tools	Dims	Source	OS	X-sc.
[Shu et al. 2017]	356,218	1	V	Hub crawl	✗	✗
[Zerouali et al. 2019]	7,380	1‡	V	Hub, Debian	✓	✗
[Liu et al. 2020]	2.2 M*	1	V	Hub crawl	✗	✗
[Ibrahim et al. 2020]	936	1	V	Hub, per-sys	✗	✗
[Zerouali et al. 2021]	140,498	1‡	V	Hub, Debian	✓	✗
[Haque et al. 2022]	261	1	V	Hub, official	✓	✗
[Dahlmanns et al. 2023]	337,171	1	S	Hub crawl	✗	✗
[Mills et al. 2023]	380	6	V	Hub categories	✗	✓
[Boles et al. 2024]	927	2	V	Hub official	✓	✓
[Bufalino et al. 2025]	20†	7	V	Hub Deb/Alp	✓	✓
[Shi et al. 2025]	33,952*	1	V,S,C,M	Hub pull-rank	✗	✗
This work	**5,606**	**14**	**V,B,S,C,M**	**Hub category**	✓	✓

Dims: V vuln, B SBOM, S secrets, C config, M malware. X-sc.: cross-scanner. $*$ vulnerability scan on a subset; † top-20 Debian/Alpine image families; ‡ Debian Security Tracker (metadata), not a scanner.

A rich line of work has charted the security of Docker Hub. [Shu et al. 2017] introduced the first large-scale study, scanning 356,218 images and showing that vulnerabilities propagate from a base image to everything built on it. [Liu et al. 2020] extended the scale to 2.2 million images; [Zerouali et al. 2019] applied the notion of *technical lag* to 7,380 Debian-based images, later generalised to a multi-dimensional analysis over 140,000 images [Zerouali et al. 2021]. [Wist et al. 2021] analysed 2,500 images across image classes, [Ibrahim et al. 2020] examined how images for the same system differ across base distributions, and the recent [Shi et al. 2025] ranked influential images by pull count at the scale of the full registry. Most relevant to us, [Haque et al. 2022] characterised the exploitability and impact of vulnerabilities in 261 official base images and reported a result we revisit in Section 4.5: highly-exploitable vulnerabilities are *more* frequent in minimal images such as Alpine, while larger distributions carry the higher-impact ones. These studies established the prevalence and propagation of vulnerabilities in the ecosystem; each draws on a single vulnerability detector, and our multi-scanner design complements them by also quantifying how much the measured posture depends on the chosen scanner. Prior scanner comparisons on focused samples report large disagreement [Boles et al. 2024, Bufalino et al. 2025], which we revisit as a secondary result at OS-base census scale (Section 4.3). Building on this body of work, we are not aware of a recent study that measures the Linux OS base images of Docker Hub specifically with a heterogeneous multi-scanner battery as a current census, nor one that quantifies the prevalence and popularity of *end-of-life* OS images that remain in heavy use. This paper addresses both.

3. Methodology

We take as our corpus the operating-system base images Docker Hub lists under its *Operating systems* category, crawled in 2026: **20** repositories with an `amd64` image (`alpine`, `ubuntu`, `debian`, `centos`, `busybox`, `amazonlinux`, `fedora`, `oraclelinux`, `rockylinux`, `almalinux`, `photon`, `archlinux`, `tumbleweed`, `leap`, `kali-rolling`, `mageia`, `alt`, `cirros`, `clearlinux`, `sl`). Deduplicating tags by `amd64` content digest (e.g. `24.04`, `noble` and `latest` are one image) leaves **5,606** unique images, the unit of measurement. An OS base image is a static root filesystem with no application source code and no running service, so we select the static-analysis axes that apply to such an artifact and run *14* open-source scanners.[9] *SBOM*: Syft and cdxgen. *Package vulnerabilities (SCA)*: Trivy, Grype, OSV-Scanner and Clair. *Image hardening*: Dockle and Checkov. *Secrets*: TruffleHog, Gitleaks, detect-secrets and Whispers. *Malware/IOC*: ClamAV and YARAHunter. We exclude the SAST, dynamic (DAST) and network axes, which need application source or a running service a base image lacks. Each image is exported once to a tar archive that all 14 scanners read, so result differences reflect the scanner rather than the pull, and raw outputs are kept verbatim.

4. Results

4.1. RQ1: Security posture by distribution

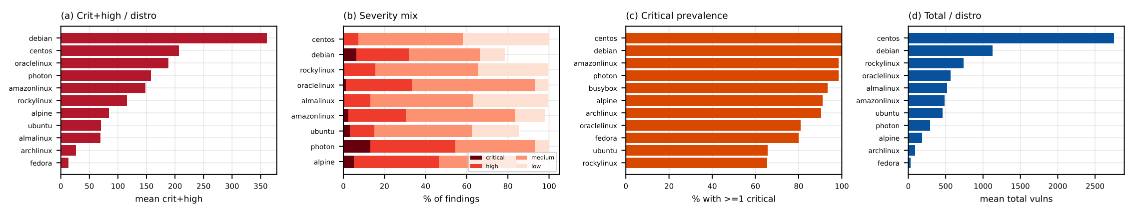

Figure 1. (a) mean critical+high per distribution; (b) severity mix; (c) critical prevalence; (d) mean total vulnerabilities.

[9] Each scanner runs from a version-pinned container image; the exact versions are recorded in the released configuration.

Critical- and high-severity vulnerability findings vary by an order of magnitude across distributions (Figure 1(a)). Debian carries the most (mean ≈344 critical+high per image, $n = 270$), followed by the RHEL family (CentOS ≈206, OracleLinux ≈189, AmazonLinux ≈152, RockyLinux ≈115); Alpine sits lower (≈84), and Arch, ALT, Kali, Mageia and openSUSE Tumbleweed report few or none. Within-distribution variance is large (Debian's SD, ≈564, exceeds its mean) and some repositories are small (CentOS $n = 10$), so these are tendencies, not tight estimates. Package count does not track this ordering: Mageia and Fedora carry many packages yet few critical or high findings (Section 4.5).

4.2. RQ2: Staleness vs. vulnerability

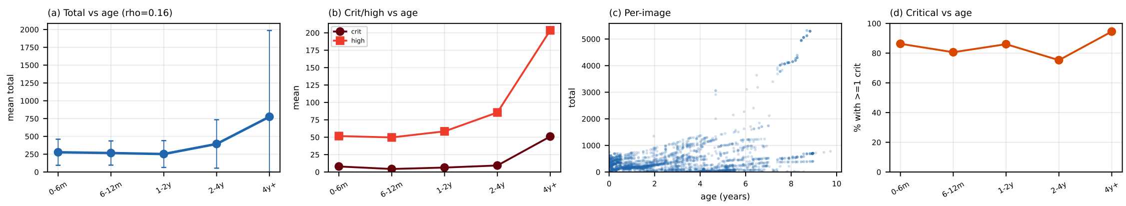

Figure 2. (a) mean total by age (±1 SD); (b) critical and high by age; (c) per-image age vs. total; (d) critical prevalence by age.

Vulnerability count tends to rise with image age (Figure 2(a)). Images rebuilt within the last six months carry a mean of ≈255 total vulnerability findings, climbing through ≈273 (one to two years) and ≈457 (two to four years) to ≈692 for images not rebuilt in over four years. The per-image rank correlation is positive but modest (Spearman $\rho = 0.16$, $p < 0.001$), reflecting wide within-bucket variance even as the bucketed means rise monotonically (the technical lag of [Zerouali et al. 2019], here across 20 distributions).

4.3. RQ3: Inter-scanner divergence on OS packages

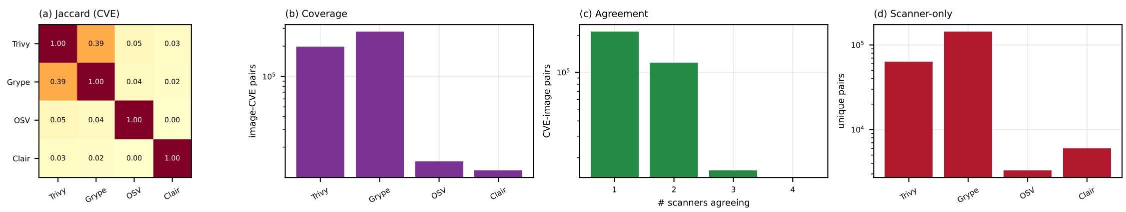

Figure 3. (a) pairwise CVE Jaccard; (b) CVEs per engine; (c) CVEs by number of agreeing engines; (d) engine-only CVEs.

The four package-vulnerability engines barely agree (Figure 3). We compare at the (image, CVE) level because the engines name packages differently (Clair reports source packages, Trivy and Grype binary packages), so a package-level key would understate agreement. Even so, the best-agreeing pair is Trivy and Grype, at a Jaccard of just **0.35**; every other pair is below 0.05, and OSV-Scanner and Clair are near-disjoint from the rest (OSV∩Clair $= 0$). The engines differ sharply in scope: Grype and Trivy report by far the most image-CVE pairs (196k and 148k), OSV-Scanner targets language-ecosystem advisories (9k) and Clair maps only specific distributions to its feeds (8k, and zero on Rocky and AlmaLinux, which it does not map to the RHEL feeds). The reported security posture of an OS image is thus, to a large degree, a property of the chosen scanner.

4.4. RQ4: End-of-life and un-pullable images

Two longevity hazards surface. First, about **one in five** of the digest-pinned OS images fail to pull on a current Docker Engine with `manifest schema unsupported`: they are old images in the deprecated Docker image-manifest schema 1, whose support modern Docker removed. The digests still exist on Docker Hub, but the images are no longer installable, and the rate varies sharply by distribution (Figure 4): over half of the `centos` and `busybox` digests and about a third of `ubuntu` are affected, against under 3% for `debian`, `archlinux` and `rockylinux`. Second, end-of-life distributions remain in heavy use: every `centos` tag has been unmaintained since June 2024, yet the repository still records over a billion pulls. To our knowledge neither the prevalence of legacy-schema OS images nor the popularity of EOL bases has been measured before.

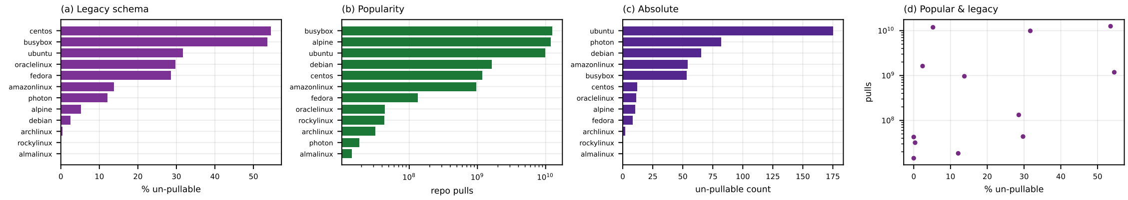

Figure 4. (a) un-pullable rate by distribution; (b) repository pulls; (c) un-pullable count; (d) rate vs. pulls.

4.5. RQ5: What drives the vulnerability load?

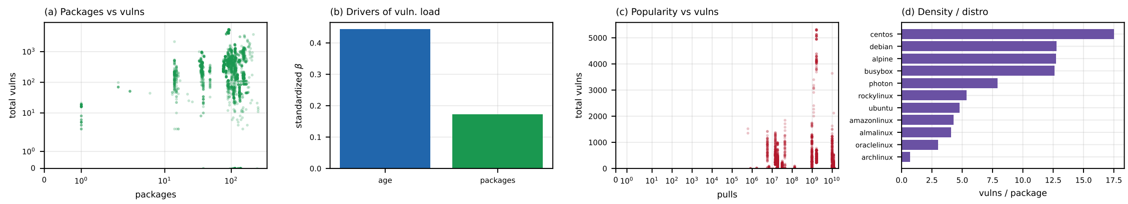

Figure 5. (a) packages vs. total; (b) standardized regression coefficients (age vs. packages); (c) popularity (pulls) vs. total; (d) vulnerabilities per package by distribution.

The intuition that a smaller image is a safer one does not hold cleanly (Figure 5(a)). The relationship between package count and vulnerability count is not monotonic: BusyBox, with a single package, carries only a handful of findings, but Mageia, with around 250 packages, carries about a dozen, while Debian carries hundreds of critical and high findings with fewer packages than Mageia. This refines [Haque et al. 2022]: on our multi-scanner data the vulnerability count is governed more by a distribution's advisory coverage and patch cadence than by image size. A multiple regression makes the ranking explicit (Figure 5(b)): with both predictors standardized, image age dominates ($\beta \approx 0.44$) over package count ($\beta \approx 0.17$), and the partial correlations agree (0.44 vs. 0.19), though a per-distribution mixed model shows distribution identity dominates both (intraclass correlation 0.53, with package count becoming a strong within-distribution driver). Popularity offers no protection either: a repository's pull count is essentially uncorrelated with its vulnerability count (Spearman $\rho = 0.00$, $p = 0.9$, Figure 5(c)), so the most-pulled bases are no safer than the rest.

4.6. Beyond vulnerabilities: secrets, misconfiguration, malware

The battery also covers three non-vulnerability axes. *Misconfiguration:* Dockle and Checkov flag a CIS-style issue in essentially every image (missing `HEALTHCHECK`, content-trust disabled, running as `root`); these are structural, so the near-universal rate is robust. *Secrets and malware:* TruffleHog/Gitleaks flag **72%** of images and YARA-Hunter **76%**, but manually reading seeded samples of **1,100** detections each found **zero** real secrets and **zero** real malware (true-positive rate $\leq 0.35\%$, 95% CI; ClamAV reports zero): every hit was a test key, package checksum, documentation example, or a YARA rule matching a benign system binary, below [Dahlmanns et al. 2023]'s 8.5% and Dr. Docker's $\sim$0.7% [Shi et al. 2025]. The validated rate on both axes is near zero against a high raw rate; as in RQ3, single-tool counts overstate, and full triage is future work.

4.7. Reproducing prior analyses

We reproduce concrete analyses from prior work on *our* OS-base corpus (Table 2); their own corpora range from the whole registry to a single distribution (the *OS?* column marks the OS-base-focused ones), yet the direction holds in every case, and the multi-scanner setting refines the minimal-image and scanner-agreement findings.

Table 2. Prior analyses reproduced on our OS-base corpus.

Study	Metric reproduced	OS?	Reported	Ours
[Shu et al. 2017]	images with a high-severity vuln	✗	>80%	97%*
[Zerouali et al. 2019]	vulnerabilities vs. image age	✓	rises with age	≈255→692 (ρ=0.16)
[Ibrahim et al. 2020]	vuln spread across images	✗	varies	10× (distro)
[Wist et al. 2021]	where severe vulns concentrate	✗	language ecosys.	OS-package layer
[Haque et al. 2022]	is a minimal image safer?	✓	no	no (non-monotonic)
[Dahlmanns et al. 2023]	images carrying secrets	✗	8.5% valid	72% raw, 0% valid
[Mills et al. 2023]	vulnerabilities vs. time; OS effect	✗	rise; Debian ≫	rise; Debian top
[Boles et al. 2024]	scanner divergence on OS images	✓	agree 15%	Jaccard 0.35
[Shi et al. 2025]	known-vulnerability prevalence	✗	93.7%	98.7%

OS?: prior study's corpus is OS base images. $*$ critical or high (Shu reports high-severity); ρ: Spearman.

5. Limitations and conclusion

Limitations. Small repositories carry wide within-distribution variance, so per-distribution means are tendencies, not tight estimates. We report raw scanner output without adjudicating false positives or negatives; the divergence of RQ3 implies no engine is ground truth, and severity rests on CVSS scores that can overstate risk without VEX or reachability data [Zhou et al. 2026]. We scan `linux/amd64` only, characterise each image as of its scan time, and cover only the *Operating systems* category; the released dataset lets others re-run the analysis under other choices. **Conclusion.** We measured the security posture of Docker Hub's Linux OS base images with 14 scanners rather than one. Posture varies by an order of magnitude across distributions and is driven by the distribution and image age rather than by image size or popularity; and about a fifth of historically-published images can no longer be installed by a modern Docker while end-of-life bases stay heavily pulled. Secondarily, the four package-vulnerability engines agree on almost nothing, so a single tool's count is largely a property of that tool. We publicly release the consolidated multi-scanner dataset and pipeline[10] so the measurement can be reproduced, audited, and extended.

[10] Released on acceptance.

References

Boles, B. et al. (2024). Deciphering discrepancies: A comparative analysis of Docker image security. In *SCAM*. IEEE.

Bufalino, J. et al. (2025). SBOMproof: Beyond alleged SBOM compliance for supply chain security of container images.

Dahlmanns, M. et al. (2023). Secrets revealed in container images: An internet-wide study on occurrence and impact. In *ASIA CCS '23*, pages 797–811. ACM.

Haque, M. U. et al. (2022). Well begun is half done: An empirical study of exploitability and impact of base-image vulnerabilities. In *SANER*, pages 1100–1111. IEEE.

Ibrahim, M. H. et al. (2020). Too many images on DockerHub! how different are images for the same system? *Empirical Softw. Eng.*, 25(5):4250–4281.

Liu, P. et al. (2020). Understanding the security risks of Docker Hub. In *Computer Security – ESORICS 2020*, volume 12308 of *Lecture Notes in Computer Science*, pages 257–276. Springer.

Mills, A. et al. (2023). Longitudinal risk-based security assessment of docker software container images. *Comput. Secur.*, 135:103478.

Shi, H. et al. (2025). Dr. Docker: A large-scale security measurement of Docker image ecosystem. In *WWW '25*. ACM.

Shu, R. et al. (2017). A study of security vulnerabilities on Docker Hub. In *CODASPY '17*, pages 269–280. ACM.

Wist, K. et al. (2021). Vulnerability analysis of 2500 Docker Hub images. In *Advances in Security, Networks, and Internet of Things*, Transactions on Computational Science and Computational Intelligence, pages 307–327. Springer.

Zerouali, A. et al. (2019). On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In *SANER 2019*, pages 491–501. IEEE.

Zerouali, A. et al. (2021). A multi-dimensional analysis of technical lag in Debian-based Docker images. *Empirical Softw. Eng.*, 26(2):19.

Zhou, L. et al. (2026). A reality check on SBOM-based vulnerability management: An empirical study and a path forward. In *CODASPY '26*. ACM.